



UNIVERSITY *of* WEST FLORIDA

Machine Learning with Scikit-learn

Transformers

Brian Jalaian, Ph.D.

Associate Professor

Intelligent Systems & Robotics Department

Table of Content

- Introduction to Transformers
- Attention Mechanisms
- Self-Attention
- Transformer Architecture
- Pretraining and Finetuning with Large Models
- Finetuning BERT in PyTorch (Lecture Code)

Introduction to Transformers

Challenges & Solutions with Traditional Sequential Models

Challenges	Solutions
RNNs/LSTMs struggle with long dependencies	Attention mechanisms
Performance degrades with sequence length	Parallel processing
Sequential processing limits speed	Direct connections

Key Innovations in Transformers

Architecture: Replace RNN with self-attention mechanism

Modeling: Direct modeling of all relationships in sequence

Benefits:

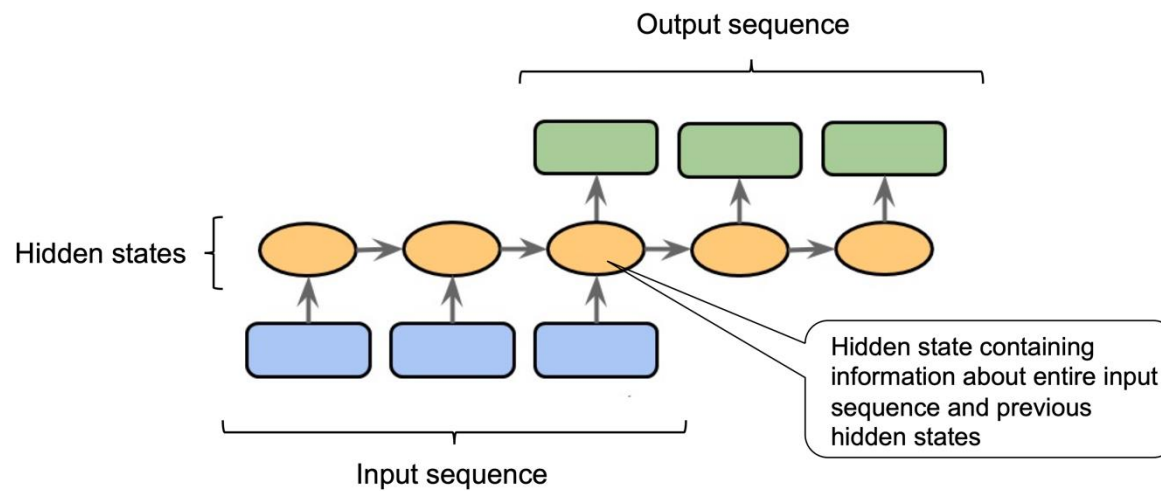
- Parallel training & computation
- Better handling of long sequences
- Improved model scalability

Attention Mechanisms

Attention Mechanisms Overview

Seq2Seq models like RNNs process an entire input sequence before producing output.

However, **compressing all information into one hidden state** may lead to information loss, especially for long sequences. Moreover, using RNN-based Seq2Seq models for translating word by word can lead to grammatical errors.



Why Attention?

Just like humans revisit parts of a sentence while translating, **Attention Mechanism** allows the model to:

- Access all input elements at each time step
- Assign different weights to each element
- Focus on relevant parts

Example: To generate "*help*"

- more focus on "*mir, helfen, zu*"
- less focus on "*kannst, du, Satz*"

German input sentence:

Kannst du mir helfen diesen Satz zu uebersetzen ?

Word-by-word translation (incorrect):

Can you me help this sentence to translate ?



German input sentence:

Kannst du mir helfen diesen Satz zu uebersetzen ?

Correct English translation:

Can you help me to translate this sentence ?



Attention for RNNs

Original paper: **Bahdanau et al., 2014** [*Neural Machine Translation by Jointly Learning to Align and Translate*](#)

Given input sequence:

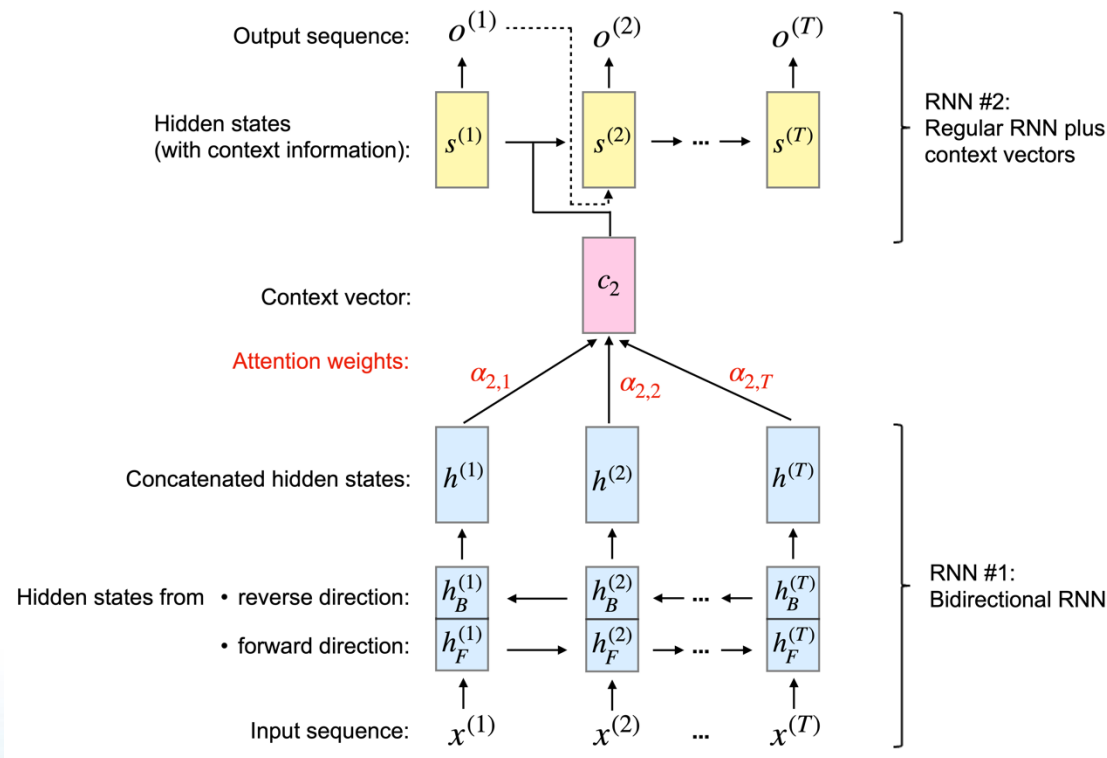
$$x = (x^{(1)}, x^{(2)}, \dots, x^{(T)})$$

- Attention assigns weight to each input element
- Helps identify which parts to focus on

For example, suppose our input is a sentence, and a word with a larger weight contributes more to our understanding of the whole sentence.

Bidirectional RNN Processing

Bidirectional RNN reads input sequence: forward ($1 \rightarrow T$) and backward ($T \rightarrow 1$). Thus, each input $x^{(i)}$ has two hidden states: $h_F^{(i)}$ (forward) and $h_B^{(i)}$ (backward). And we can concatenate them: $h^{(i)} = [h_F^{(i)}, h_B^{(i)}]$ for computation efficiency.

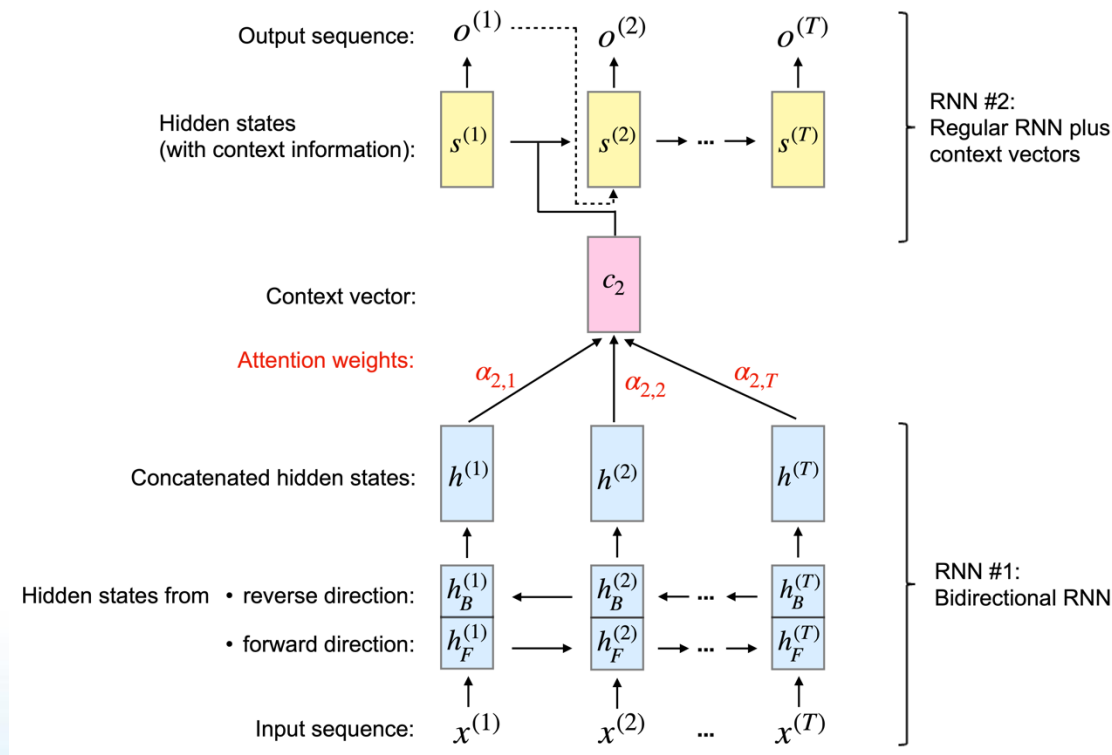


Bidirectional RNN Processing (Cont'd)

Context vector c_i is a weighted sum of all hidden states:

$$c_i = \sum_{j=1}^T \alpha_{ij} h^{(j)}$$

Where α_{ij} is attention weight, and each output has a unique set of α_{ij} .



Bidirectional RNN Processing (Cont'd)

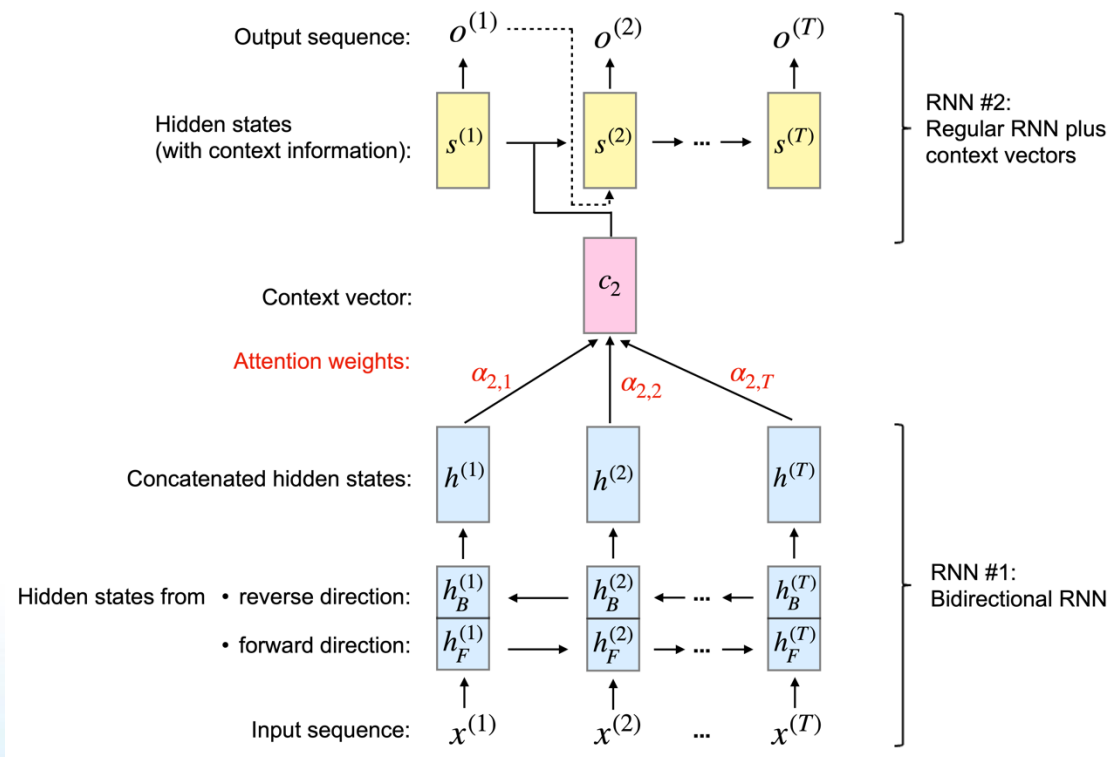
Output Generation

At time step i , RNN#2 uses:

- Previous hidden state $s^{(i-1)}$
- Previous target word $y^{(i-1)}$
- Context vector $c^{(i)}$

To generate output $o^{(i)}$

- During training $\rightarrow$ feed correct $y^{(i)}$
- During inference $\rightarrow$ feed predicted $o^{(i)}$



Bidirectional RNN Processing (Cont'd)

Computing Attention Weights

The attention weight α_{ij} is a normalized version of the alignment score e_{ij} , where the alignment score evaluates how well the input around position j matches with the output at position i . Each weight α_{ij} has:

- j : index of input
- i : index of output

Normalized alignment score:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^T \exp(e_{ik})}$$

Similar to **Softmax** function, estimating class probabilities in multiclass classification via the softmax function, and thus, $\sum_{j=1}^T \alpha_{ij} = 1$.

RNN Attention vs Transformer Attention



RNN Attention:

- Uses attention + recurrent process (one element at a time)

Transformer Attention:

- Uses only self-attention
- Processes whole sequence at once
- No recurrence

Self-Attention

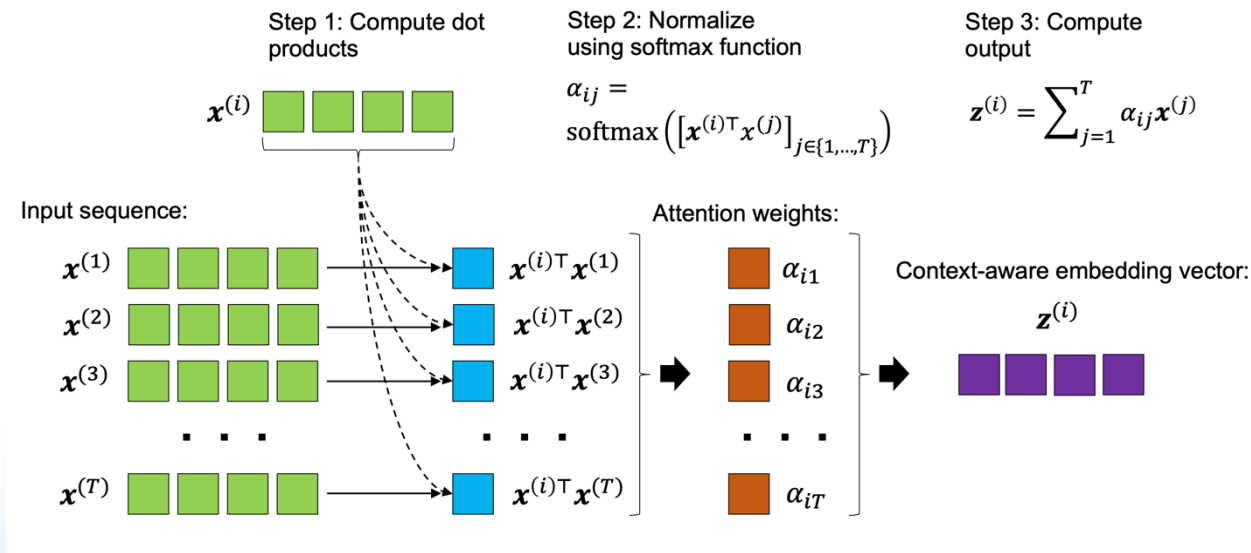
Basic Self-Attention

Setup:

- Input sequence: $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(T)}$
- Output sequence (self-attention layer): $\mathbf{z}^{(1)}, \mathbf{z}^{(2)}, \dots, \mathbf{z}^{(T)}$
- Final model output: $\mathbf{O}$

Each input $\mathbf{x}^{(i)}$ and output $\mathbf{z}^{(i)}$:

- Vector of size $d \rightarrow \mathbf{x}^{(i)} \in \mathbb{R}^d$



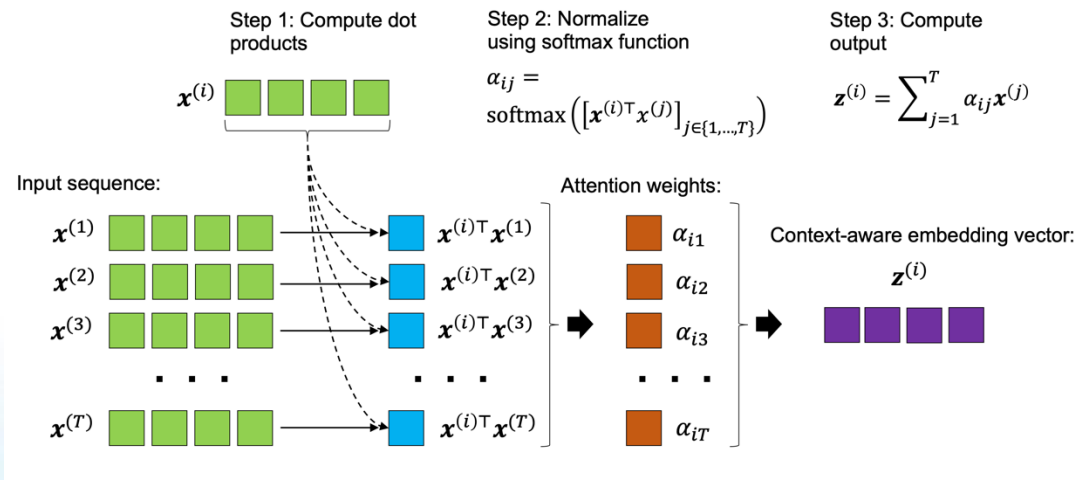
Goal of Self-Attention

For Seq2Seq tasks:

- Model dependencies between current element and **all other elements**
- Without recurrence (no RNN)

Three key steps:

1. Compute similarity-based importance weights
2. Normalize weights (Softmax)
3. Use weights to compute context-aware embeddings



Self-Attention Computation

Each output $\mathbf{z}^{(i)}$ (context-aware embedding vector) is a weighted sum of inputs:

$$\mathbf{z}^{(i)} = \sum_{j=1}^T \alpha_{ij} \mathbf{x}^{(j)}$$

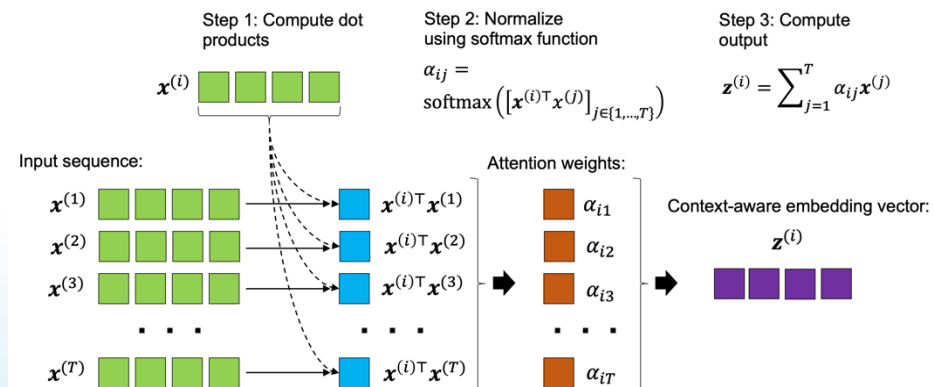
Attention weights α_{ij} :

1. Measure similarity between $\mathbf{x}^{(i)}$ and $\mathbf{x}^{(j)}$, which is the “raw” weight: $\omega_{ij} = \mathbf{x}^{(i)\top} \mathbf{x}^{(j)}$
2. Normalize ω_{ij} via the softmax function:

$$\alpha_{ij} = \frac{\exp(\omega_{ij})}{\sum_{j=1}^T \exp(\omega_{ij})} = \text{softmax}([\omega_{ij}]_{j=1\dots T})$$

Similar to **Softmax** function, estimating class probabilities in multiclass classification via the softmax function:

$$\sum_{j=1}^T \alpha_{ij} = 1.$$



Scaled Dot-Product Attention

To make self-attention **learnable & flexible**, we introduce three trainable weight matrices: $\mathbf{U}_q$, $\mathbf{U}_k$, and $\mathbf{U}_v$.

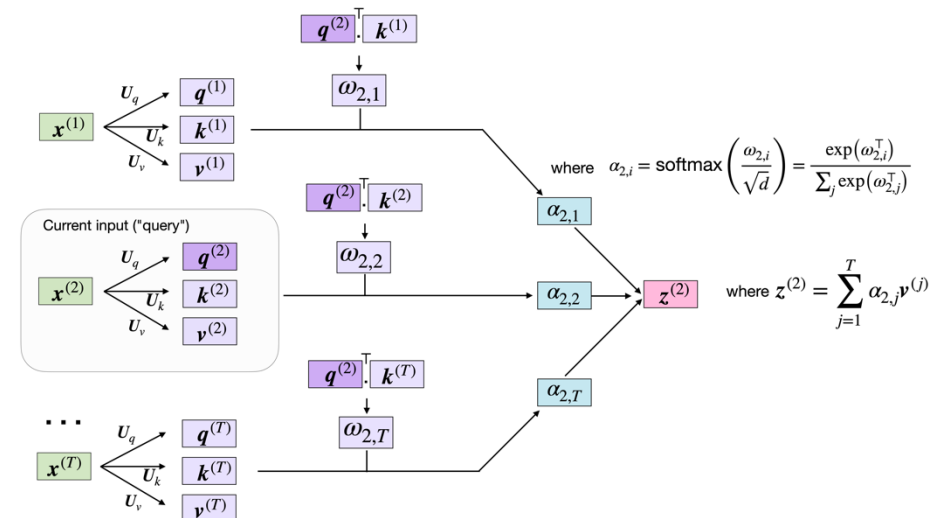
Query-Key-Value Representation

Project inputs to:

- Query sequence: $\mathbf{q}^{(i)} = \mathbf{U}_q \mathbf{x}^{(i)}$ for $i \in [1, T]$
- Key sequence: $\mathbf{k}^{(i)} = \mathbf{U}_k \mathbf{x}^{(i)}$ for $i \in [1, T]$
- Value sequence: $\mathbf{v}^{(i)} = \mathbf{U}_v \mathbf{x}^{(i)}$ for $i \in [1, T]$

Dimensions:

- d : input vector size
- d_k : dimension of query/key
- d_v : dimension of value
- $\mathbf{U}_q, \mathbf{U}_k \in \mathbb{R}^{d_k \times d}$
- $\mathbf{U}_v \in \mathbb{R}^{d_v \times d}$



Scaled Dot-Product Attention (Cont'd)

Unnormalized attention weight:

$$\omega_{ij} = \mathbf{q}^{(i)\top} \mathbf{k}^{(j)}$$

Normalized attention weights (scaled):

$$\alpha_{ij} = \text{softmax}\left(\frac{\omega_{ij}}{\sqrt{m}}\right)$$

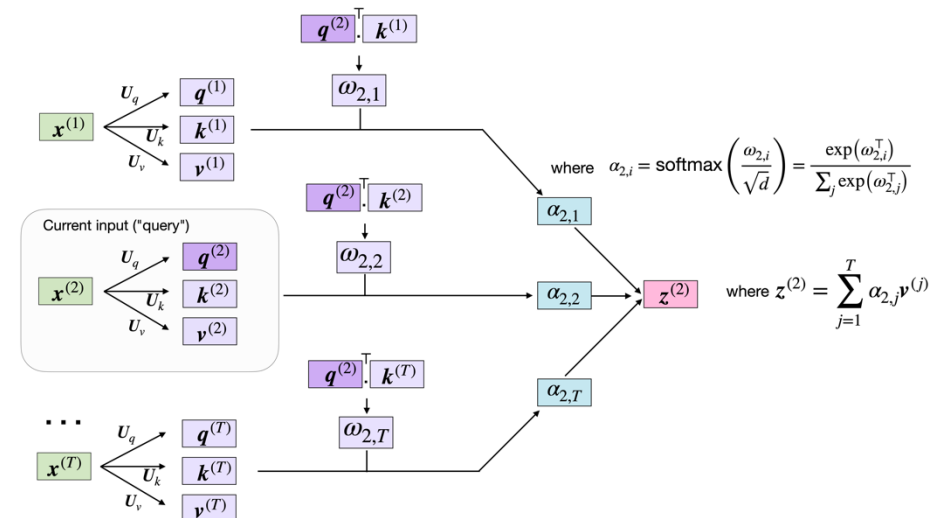
- Typically, $m = d_k$.

Why scaling? Scaling by $1/\sqrt{m}$ keeps weight vector lengths stable.

The context-aware embedding: $\mathbf{z}^{(i)} = \sum_{j=1}^T \alpha_{ij} \mathbf{v}^{(j)}$

Summary

Self-attention $\rightarrow$ flexible, learnable mechanism to model global dependencies without recurrence.



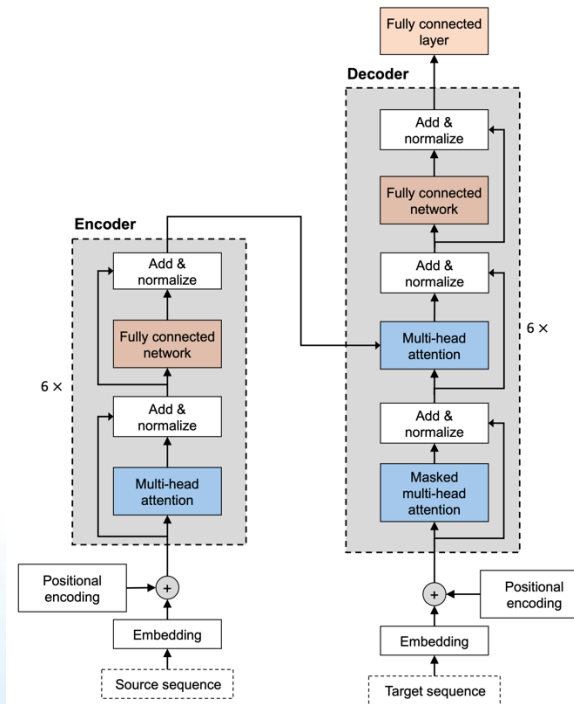
Transformer Architecture

Transformer Structure

Thanks to **self-attention**, a transformer can capture **long-range dependencies** in sequences. In NLP, this improves understanding of sentence structure and meaning.

Components:

- **Encoder** → Maps input sequence to continuous representations
- **Decoder** → Generates output sequence using encoder outputs and previous outputs

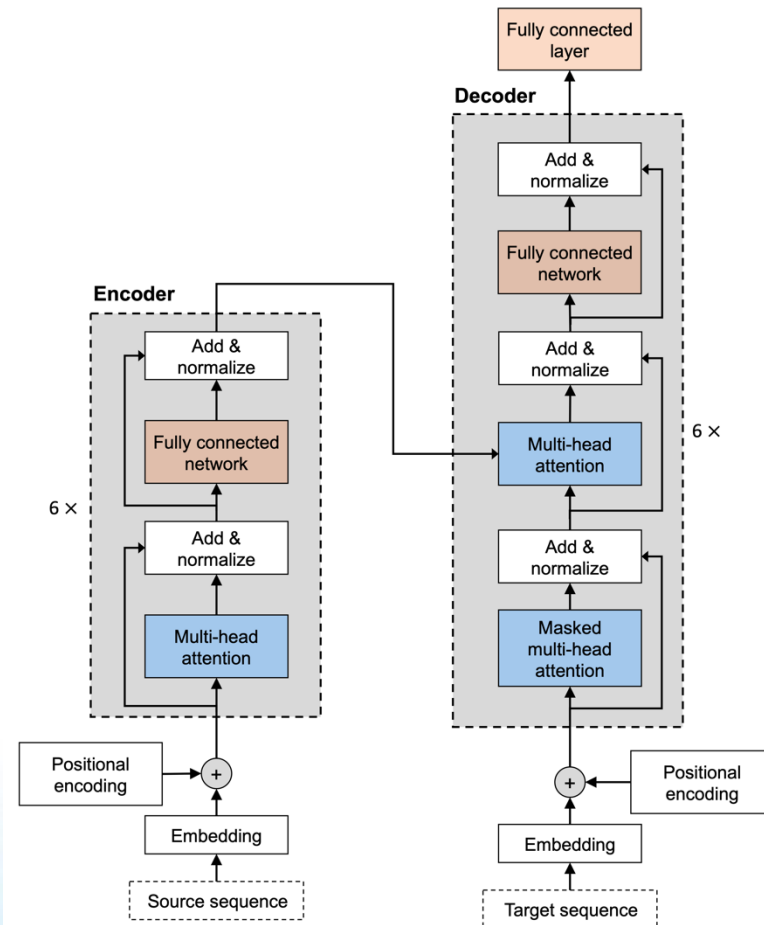


Encoder: Context Embedding

- Input: $\mathbf{X} = (\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(T)})$
- Output: $\mathbf{Z} = (\mathbf{z}^{(1)}, \mathbf{z}^{(2)}, \dots, \mathbf{z}^{(T)})$

The encoder has 6 identical layers, each with:

1. Multi-head self-attention
2. Fully connected feed-forward layer



Multi-Head Self-Attention

Each attention head has three projection matrices:

- $U_{q_j} \rightarrow$ Query
- $U_{k_j} \rightarrow$ Key
- $U_{v_j} \rightarrow$ Value

For h heads, each matrix shape:

- $U_{q_j}, U_{k_j} \in \mathbb{R}^{d_k \times d}$
- $U_{v_j} \in \mathbb{R}^{d_v \times d}$
- Often, $d_k = d_v = m$

For input $\mathbf{x}^{(i)}$ in head j : $\mathbf{q}_j^{(i)} = U_{q_j} \mathbf{x}^{(i)}$

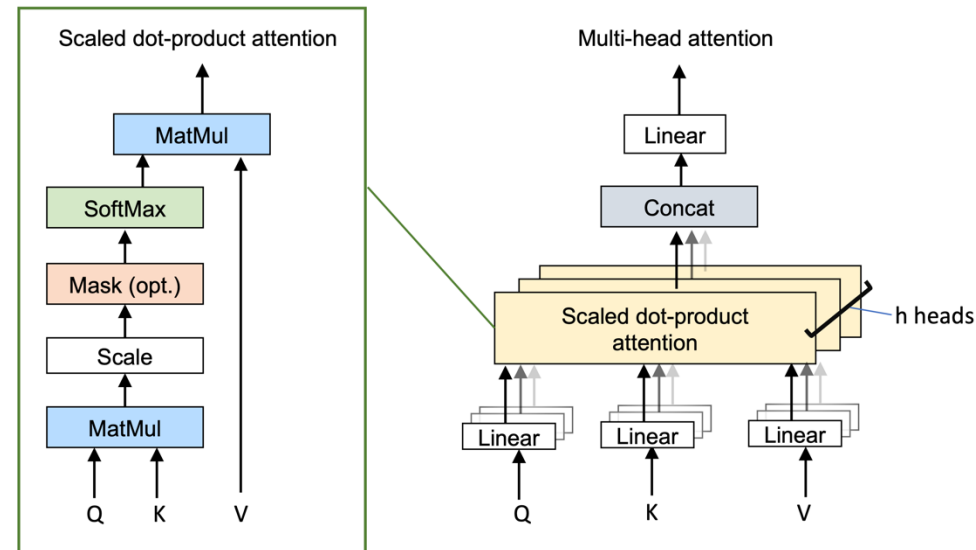
Repeat for all heads $j \in \{1, \dots, h\}$.

Summary:

Multiple attention heads compute scaled dot-product attention in parallel.

This allows the model to capture different aspects of the input sequence.

Computation is parallelizable due to no dependencies between multiple heads.



Fully Connected Layer

Each encoder layer also includes a feed-forward network:

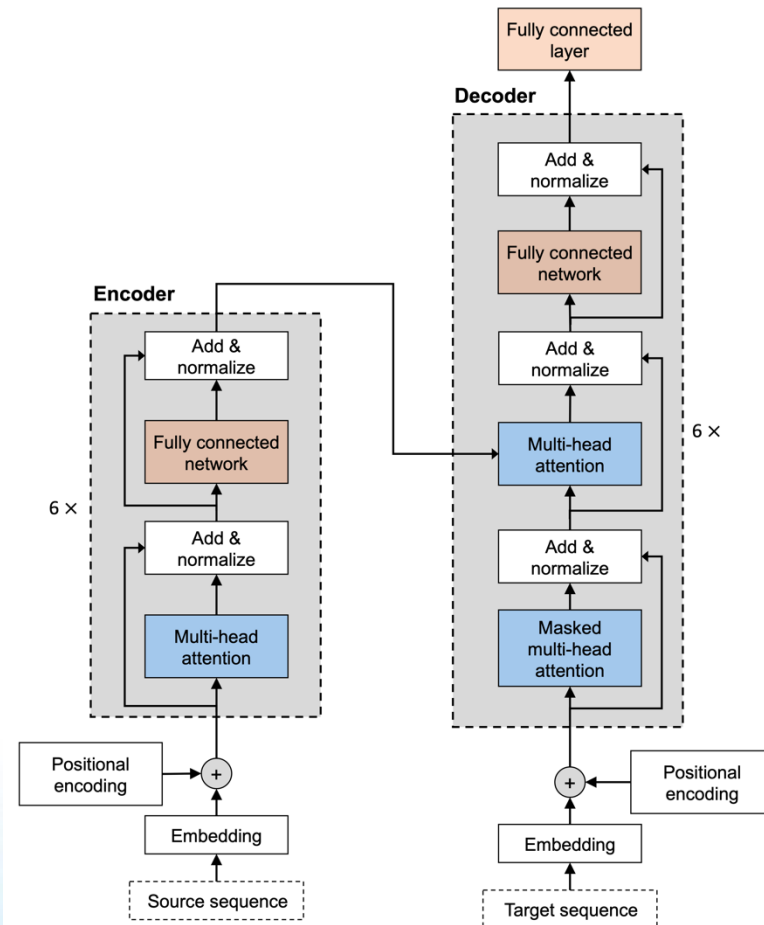
$$\text{FFN}(x) = \text{ReLU}(\mathbf{W}_1 x + b_1) \mathbf{W}_2 + b_2$$

Properties:

Same transformation applied to all input tokens

Each layer has different $\mathbf{W}_1$, $\mathbf{W}_2$, b_1 , b_2

Each sublayer has a residual connection



Decoder: Masked Multi-Head Attention

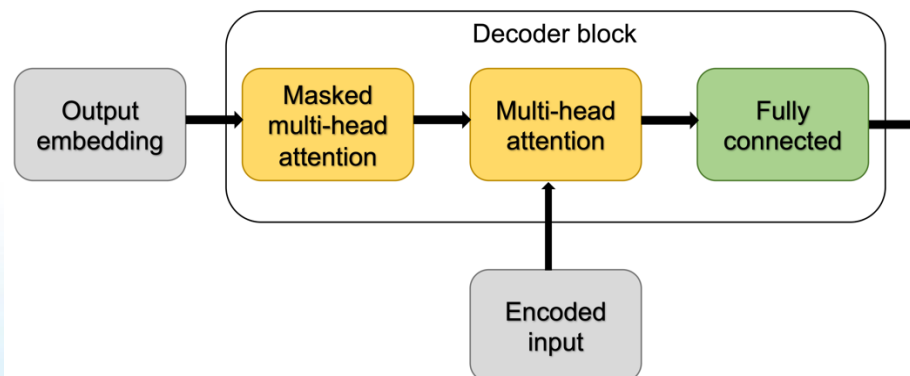
Masked attention:

Limits input sequence by **masking future positions** during training. At position i , only positions $1, 2, \dots, i - 1$ are used. For example,

$$\text{mask}(\mathbf{QK}^T) = \begin{bmatrix} e_{11} & -\infty & \dots & -\infty \\ e_{21} & e_{22} & \dots & -\infty \\ \vdots & \vdots & \ddots & \vdots \\ e_{m1} & e_{m2} & \dots & e_{mn} \end{bmatrix}$$

Decoder Process

1. Masked multi-head attention on previous outputs
2. Multi-head attention combining encoder output and decoder output
3. Feed-forward layer to generate final output probabilities



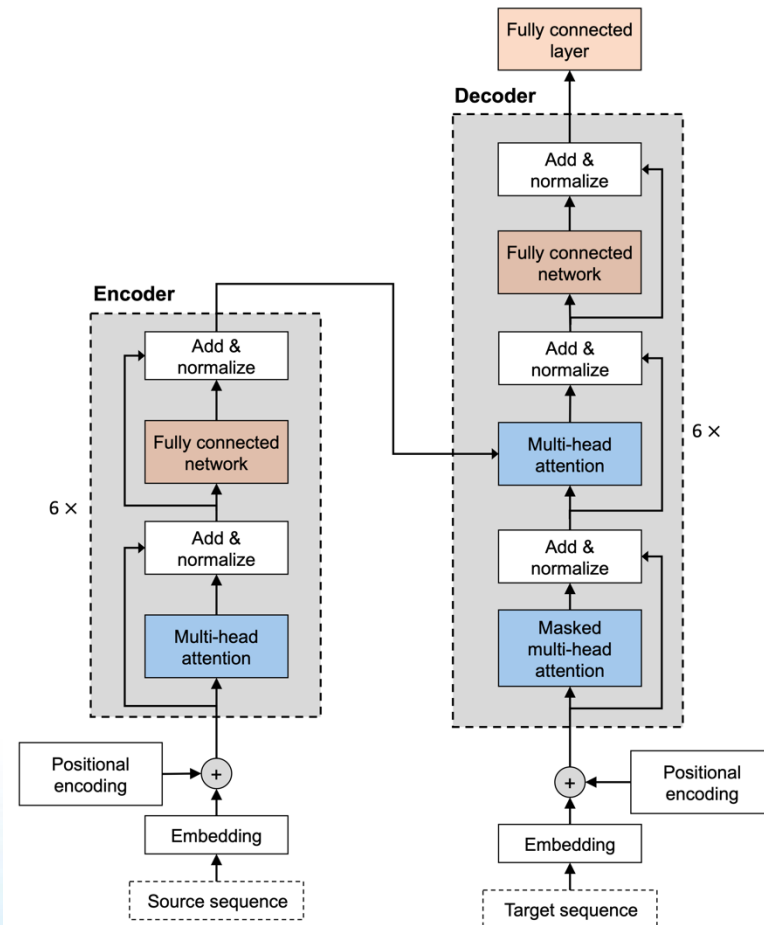
Encoder vs Decoder Attention

Encoder:

- Attention over all input elements (bidirectional)

Decoder:

- Encoder output (bidirectional)
- Previous output elements only (unidirectional)



Positional Encodings

Why needed?

Attention layers are **permutation-invariant** → sequence order is ignored without positional encoding.

For example, "Mary gives John a flower" $\neq$ "John gives Mary a flower"

- The words occurring in the two sentences are exactly the same; the meanings, however, are very different.

Sinusoidal Positional Encoding (prevent large positional values)

- $PE_{(i,2k)} = \sin\left(\frac{pos}{10000^{2k/d_{model}}}\right)$
- $PE_{(i,2k+1)} = \cos\left(\frac{pos}{10000^{2k/d_{model}}}\right)$
 - i : word position
 - k : encoding dimension
 - Encoding dimension = word embedding dimension

Types of Positional Encoding

1. **Absolute:** Records absolute word positions (sensitive to word shifts)
2. **Relative:** Captures relative positions only (invariant to sentence shift)

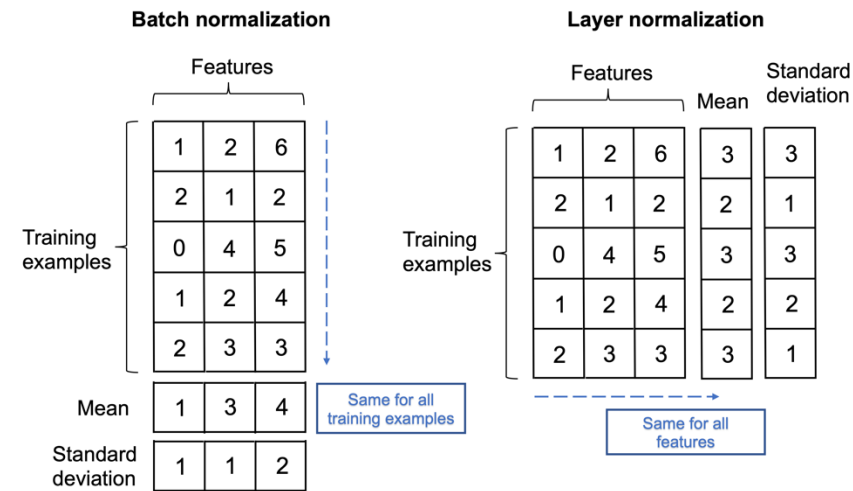
Layer Normalization

Preferred over batch normalization in NLP:

- Works with variable-length sentences
- No dependency on minibatch size

Comparison:

- **Batch Norm:** Uses batch statistics
- **Layer Norm:** Uses statistics per example



In transformers:

- Sentences padded to same length
- No recurrence → no dependency on past inputs

Pretraining and Finetuning with Large Models

The Data-Hungry Problem

The transformer architecture works well for language translation but requires:

- Large labeled datasets
- Supervised training, which is costly and data-hungry

Obtaining large labeled datasets is expensive and often impractical.

However: Large volumes of **unlabeled text** exist (books, websites, social media)

Data Hungry Solution: Self-Supervised Learning



Self-supervised learning (a.k.a. **unsupervised pre-training**): Generate “labels” directly from text.

- For example, given a large, unlabeled text corpus, we train the model to perform **next-word prediction**, which enables the model to learn the probability distribution of words and can form a strong basis for becoming a powerful language model.

Objective: Train the model to learn the probability distribution of words.

Next-Word Prediction

Given a sentence of n words:

1. At time step i , feed in ground-truth words $1, \dots, i - 1$
2. Predict word at position i
3. Compare prediction with ground-truth
4. Update model and repeat until all words processed

Note: Each iteration uses the correct previous words (ground-truth), not model predictions.

Pretraining & Finetuning Workflow

Two-stage training:

1. **Pre-training**

1. Large, **unlabeled** dataset
2. Model learns **general language representations**

2. **Fine-tuning**

1. **Labeled dataset**
2. Model adapted for **specific downstream tasks**

Pre-trained Model Representations

Pre-trained model learns representations:

- Think of these **representations** as **hidden layer activations** from the final layers.
- These representations can be used in 2 ways:

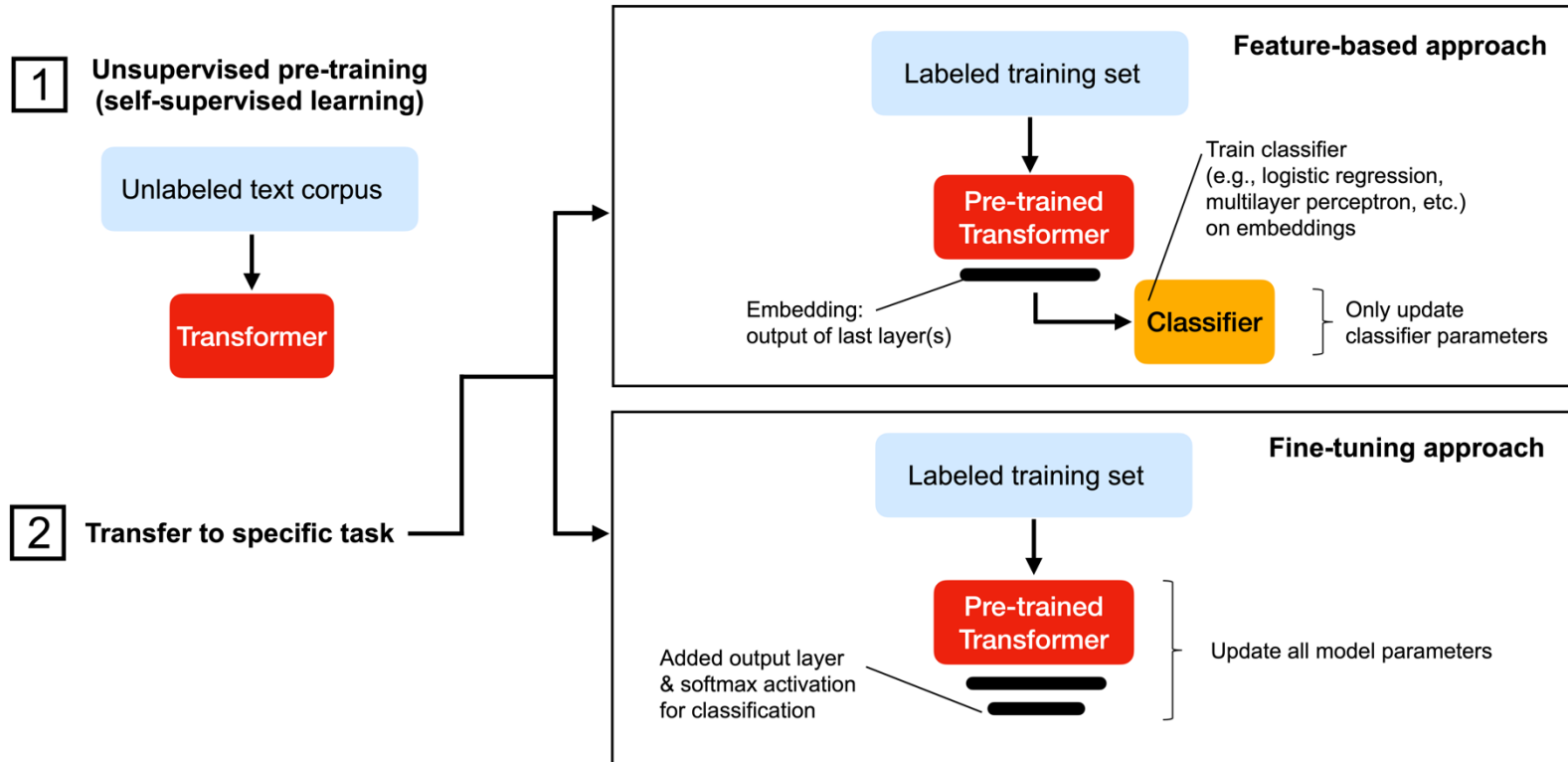
1. Feature-Based Approach

- Use **pre-trained representations** as additional features to a labeled dataset.
Requires model to learn how to extract sentence features.
- A well-known approach is **ELMo** (Embeddings from Language Models).

2. Fine-Tuning Approach

Fine-tune entire pre-trained model on a labeled dataset:

- Add an additional fully connected layer
- Train via backpropagation
- Adapt to specific task (e.g., classification)
- A well-known example is **BERT** → Pre-trained as a bidirectional language model

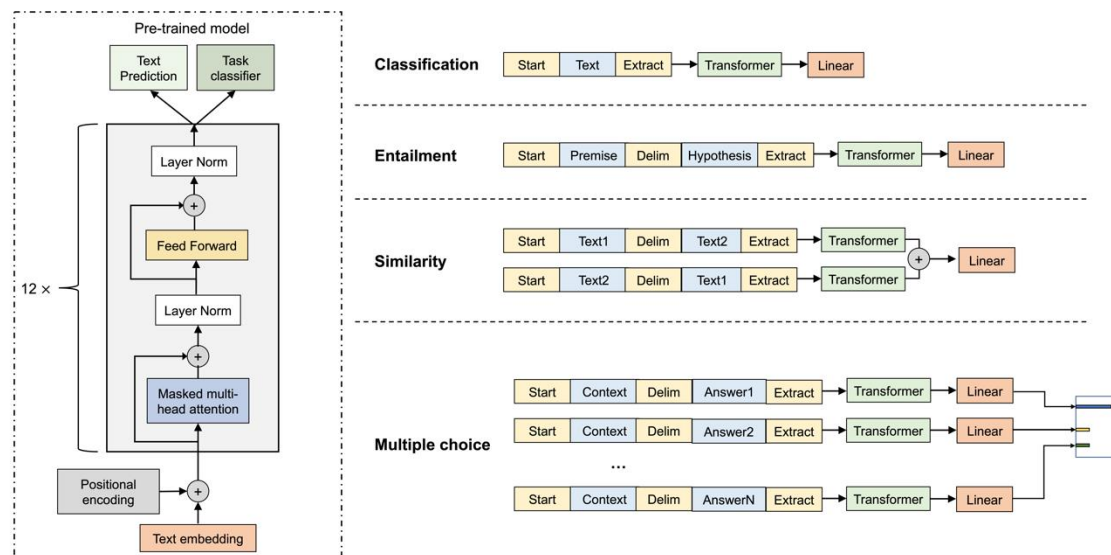


Generative Pre-trained Transformer (GPT)

We can consider **GPT-1** as a transformer model composed of:

1. A decoder-only structure (no encoder block)
2. An additional layer added later for supervised fine-tuning on specific tasks

During pre-training, GPT-1 uses the transformer decoder structure, which enables it to predict the next word based only on the preceding words. This corresponds to **unidirectional self-attention**, where each token can attend only to its left-hand context. This architecture is suitable for text generation, where words are generated one by one in a left-to-right fashion.



GPT-1 Training and Inference

- During **training**, the model receives the correct tokens from previous positions as input for predicting the next token.
- During **inference**, the model must rely on the tokens it has already generated to predict the next one.

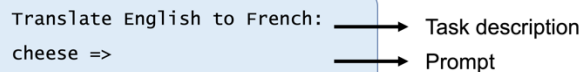
Phase	Input Source
Training	Ground truth tokens
Inference	Model-generated tokens (autoregressive)

From GPT-1 to GPT-2

The three settings we explore for in-context learning

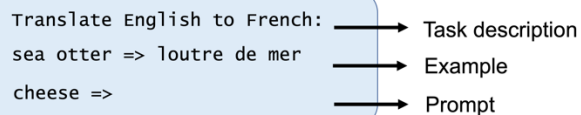
1) Zero-shot

The model is only given a natural language description of the task. No weight updates are performed.



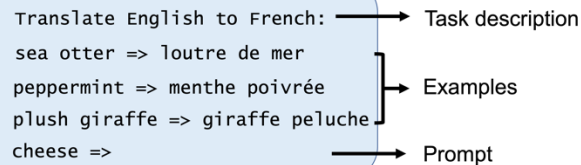
2) One-shot

In addition to the task description, the model is also given a simple example of a task. No weight updates are performed.



3) Few-shot

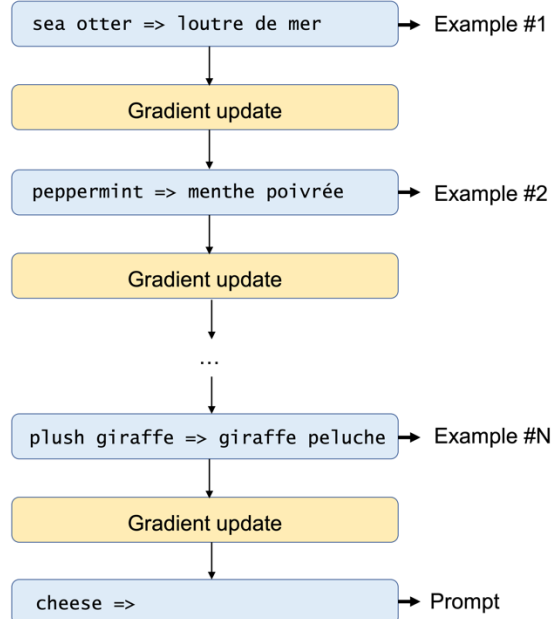
In addition to the task description, the model is also given a simple example of a task. No gradient updates are performed.



Traditional fine-tuning (not for GPT-3)

Fine-tuning

The model is trained using a large corpus of example tasks.



GPT-2 improves upon GPT-1 by eliminating the need for explicit task formatting or fine-tuning during downstream usage. This is referred to as the **zero-shot setting**, where:

- The model can handle tasks without fine-tuning
- It relies on minimal context or hints embedded in the input to infer the task

Key Ideas:

- Output is conditioned on both input and task type, i.e., $p(\textit{output} \mid \textit{input}, \textit{task})$
- This contrasts with earlier models where the output depended only on the input

Example:

If the input contains a phrase like `Translate to French: English text`, GPT-2 can infer that the task is **machine translation**, and generate the corresponding French output, even without explicit task-specific training.

GPT-3: Few-Shot and In-Context Learning



GPT-3 takes a slightly different direction by emphasizing **in-context learning** through:

- **One-shot learning:** One example provided in the prompt
- **Few-shot learning:** A few examples provided
- **In-context learning:** The model learns the pattern based on examples embedded in the input prompt, without updating model weights

Compared to GPT-2's pure zero-shot approach, GPT-3 acknowledges that:

- Providing no examples may be too difficult
- Few-shot learning aligns more with human learning, where people usually need to see a few examples before generalizing to new tasks.

GPTs Comparison

Model	Structure	Input Handling	Learning Style	Task Formatting
GPT-1	Decoder-only	Requires fine-tuning	Supervised + Pretraining	Task-specific formatting
GPT-2	Decoder-only	Infers tasks from context	Zero-shot	Minimal hints in context
GPT-3	Decoder-only	Uses examples in prompts	One-shot / Few-shot	In-context learning

Bidirectional Encoder Representations from Transformers (BERT)



It was introduced by a Google research team in 2018 and contains 345 million parameters. BERT's core innovation lies in its bidirectional (or nondirectional) training, which allows it to read and understand all tokens in an input sequence simultaneously. This means the encoding of each word takes into account **both the preceding and succeeding words**, unlike unidirectional models like GPT.

Bidirectional training enhances the quality of the input encodings, making BERT particularly effective for tasks like **classification**. However, this also means BERT cannot generate sentences word by word in an autoregressive manner.

BERT Encodings

In the BERT encoder, token representations are formed from three components:

- Token Embeddings
- Positional Embeddings
- Segment Embeddings — indicate sentence membership, particularly important for BERT's next-sentence prediction pre-training task.

These three components are summed together to produce the input embeddings used by the encoder.

Input	[CLS]	my	dog	is	cute	[SEP]	he	likes	play	##ing	[SEP]
Token embeddings	$E_{[CLS]}$	E_{my}	E_{dog}	E_{is}	E_{cute}	$E_{[SEP]}$	E_{he}	E_{likes}	E_{play}	$E_{##ing}$	$E_{[SEP]}$
	+	+	+	+	+	+	+	+	+	+	+
Segment embeddings	E_A	E_A	E_A	E_A	E_A	E_A	E_B	E_B	E_B	E_B	E_B
	+	+	+	+	+	+	+	+	+	+	+
Position embeddings	E_0	E_1	E_2	E_3	E_4	E_5	E_6	E_7	E_8	E_9	E_{10}

BERT Pre-training

Pre-training

BERT pre-training includes two unsupervised tasks:

1. Masked Language Modeling (MLM)
2. Next-Sentence Prediction (NSP)

The training objective of BERT is to **minimize the combined loss function** of both tasks.

BERT: Masked Language Modeling (MLM)



In the MLM task, tokens in the input sequence are randomly replaced with a special [MASK] token, and the model is trained to predict the original token that was masked. This is analogous to a “fill in the blanks” task.

Unlike GPT’s next-word prediction, MLM enables the model to attend to **both left and right contexts**, except for the masked tokens.

To prevent inconsistencies between pre-training and fine-tuning—since [MASK] tokens do not appear in real downstream tasks—BERT employs a masking strategy on 15% of tokens in each sequence, further divided as follows:

BERT: Masked Language Modeling (MLM)

- Keeping the word unchanged helps the model learn the original token's representation in context.
- Random word substitution prevents the model from overfitting by memorizing patterns.
- These probabilities were chosen based on ablation studies (referenced from the GPT-2 paper).

Strategy	Probability
Replace with [MASK]	80%
Replace with a random word	10%
Keep the original word unchanged	10%

BERT: Masked Language Modeling (MLM)

Example:

In the sentence “*The quick brown fox jumps over the lazy dog*”, the word *fox* might:

- be replaced with [MASK]
- be replaced with a random word like *coffee*
- remain unchanged
- The model must then predict that the correct word is *fox*.

Input sentence: A quick brown fox jumps over a lazy dog.

Output sentence: {

- 80% mask tokens: replace fox with [MASK]
- 10% random tokens: replace fox with coffee
- 10% unchanged tokens: keep fox

BERT: Next-Sentence Prediction (NSP)



Many NLP tasks, such as question answering, rely on understanding the relationship between two sentences. In NSP, the model is given a pair of sentences in the following format:

[CLS] Sentence A [SEP] Sentence B [SEP]

- [CLS] is a special classification token that appears at the beginning of the sequence and is used to predict the NSP label.
- [SEP] is a separator token indicating the end of a sentence.

The model must classify whether Sentence B is the actual next sentence following Sentence A.

- 50% of examples are true next sentences (“IsNext”).
- 50% are randomly selected, unrelated sentences (“NotNext”).
- This task helps BERT learn inter-sentence relationships and improves performance on document-level understanding tasks.

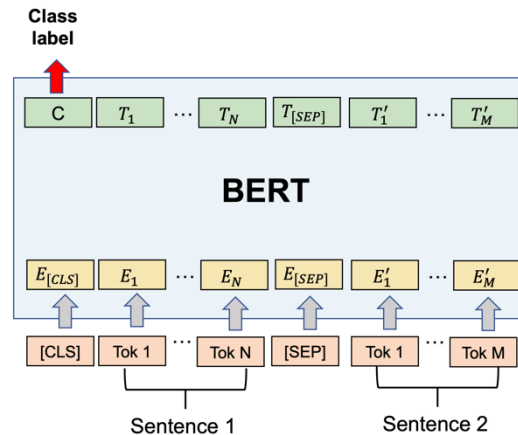
BERT: Fine-Tuning

Each input example needs to match a certain format; for example, it should begin with a [CLS] token and be separated using [SEP] tokens if it consists of more than one sentence.

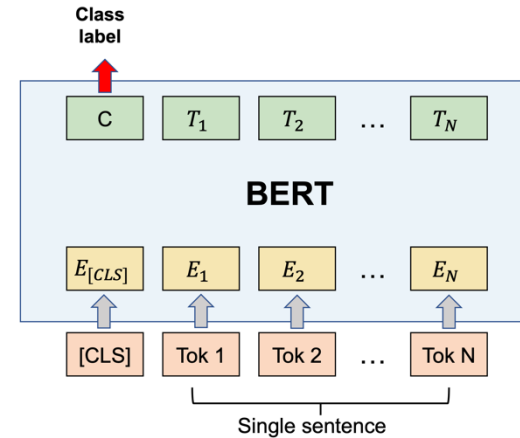
- Roughly speaking, BERT can be fine-tuned on four categories of tasks:
- **Sequence-level:** (a) sentence pair classification (b) single-sentence classification
 - It only require an additional softmax layer to be added to the output representation of the [CLS] token.
- **Token-level:** (c) question answering (d) single-sentence tagging
 - The model passes output representations of all related tokens to the softmax layer to predict a class label for each individual token.

The model fine-tuning setup has a very simple structure: an input encoder is attached to a pre-trained BERT, and a softmax layer is added for classification. Once the model structure is set up, all the parameters will be adjusted along the learning process.

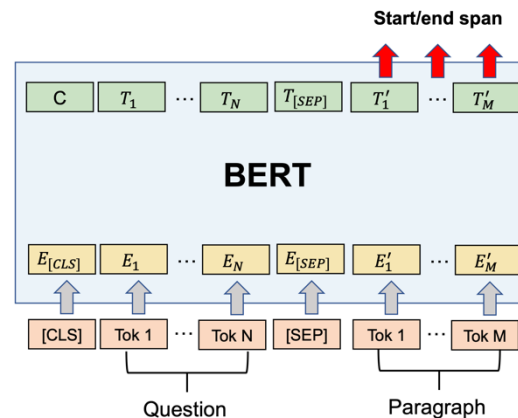
BERT: Fine-Tuning



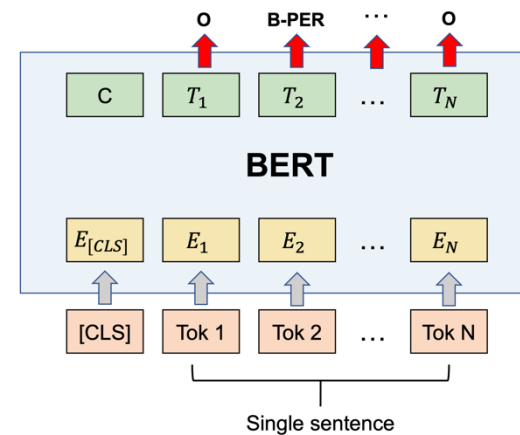
(a) Sentence pair classification tasks



(b) Single sentence classification tasks



(c) Question answering tasks



(d) Single sentence tagging tasks

Bidirectional and Auto-Regressive Transformers (BART)



- **BART** stands for **Bidirectional and Auto-Regressive Transformer**.
- Developed by Facebook AI Research (FAIR) in 2019.
- Reference: *Lewis et al., 2019* - [BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension](#).
- BART achieved state-of-the-art results on multiple generation tasks: **Abstractive QA, Dialogue Response, Summarization**. Also competitive in classification tasks, similar to BERT.
- Combines **BERT's bidirectional encoder** with **GPT's autoregressive decoder**.

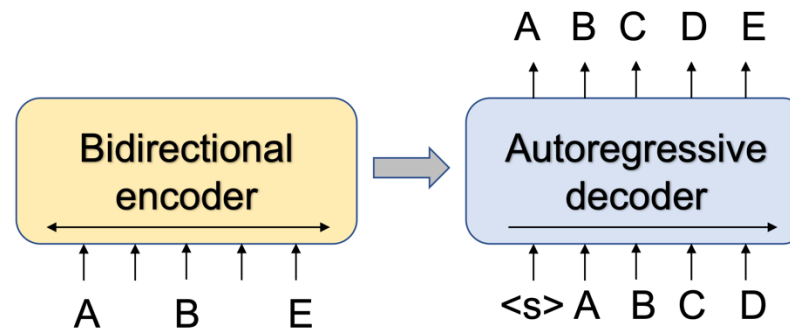
Model	Structure Used	Task Strength
BERT	Encoder-only	Classification
GPT	Decoder-only	Text Generation
BART	Encoder + Decoder	Both Generation and Classification

BART Architecture

- **Encoder:** Fully bidirectional (like BERT).
- **Decoder:** Autoregressive (left-to-right, like GPT).

Training Flow:

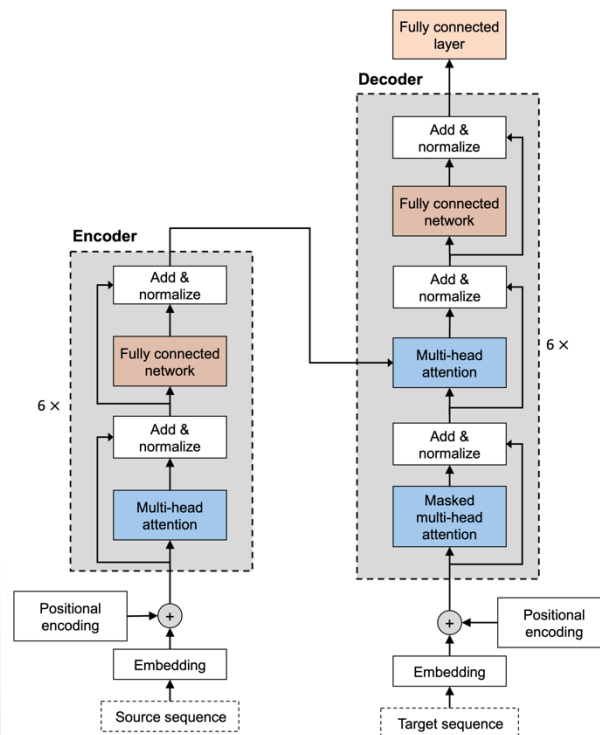
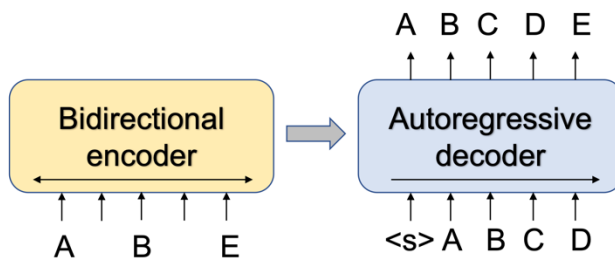
1. Input text is "corrupted" (i.e., altered in some way).
2. Corrupted input is encoded.
3. Decoder receives encoder output + generated tokens.
4. Model is trained to reconstruct the original text using cross-entropy loss.



BART Architecture

Differences from Original Transformer

- Original Transformer used **source** + **target** sequences (for translation tasks).
- BART generalizes the input format:
 - Uses only a source sequence as input.
 - Target sequence is used only during training to compute loss, not fed to decoder directly.
- Minor architectural differences (e.g., activation functions, model size tweaks).



BART Pre-training: Corruption Strategies



BART corrupts the input text using several techniques:

1. **Token Masking** – mask selected tokens.
 2. **Token Deletion** – remove some tokens.
 3. **Text Infilling** – remove spans and fill them in.
 4. **Sentence Permutation** – reorder sentences.
 5. **Document Rotation** – rotate entire text (starting at random point).
- One or more techniques may be applied at once.
 - If corruption is extreme, encoder becomes less effective → model behaves more like decoder-only GPT.

BART Fine-Tuning Tasks

(a) Sequence Classification

- Add a special token (like [CLS] in BERT) for classification.
- Use uncorrupted input (not corrupted like in pre-training).
- Input passed to both encoder and decoder.

(b) Token Classification

- No need for additional tokens.
- Use per-token hidden representations from decoder for classification.

(c) Sequence Generation

- Use for summarization or abstractive QA.
- Encoder takes in full context; decoder generates output autoregressively.

(d) Machine Translation

- Due to similarity with original transformer.
- Fine-tuning procedure:
 1. Add a new encoder with randomly initialized weights.
 2. **Step 1:** Freeze pretrained BART and train the new encoder.
 3. **Step 2:** Unfreeze and fine-tune the entire model.

Go to Lecture Code

Finetuning BERT in PyTorch